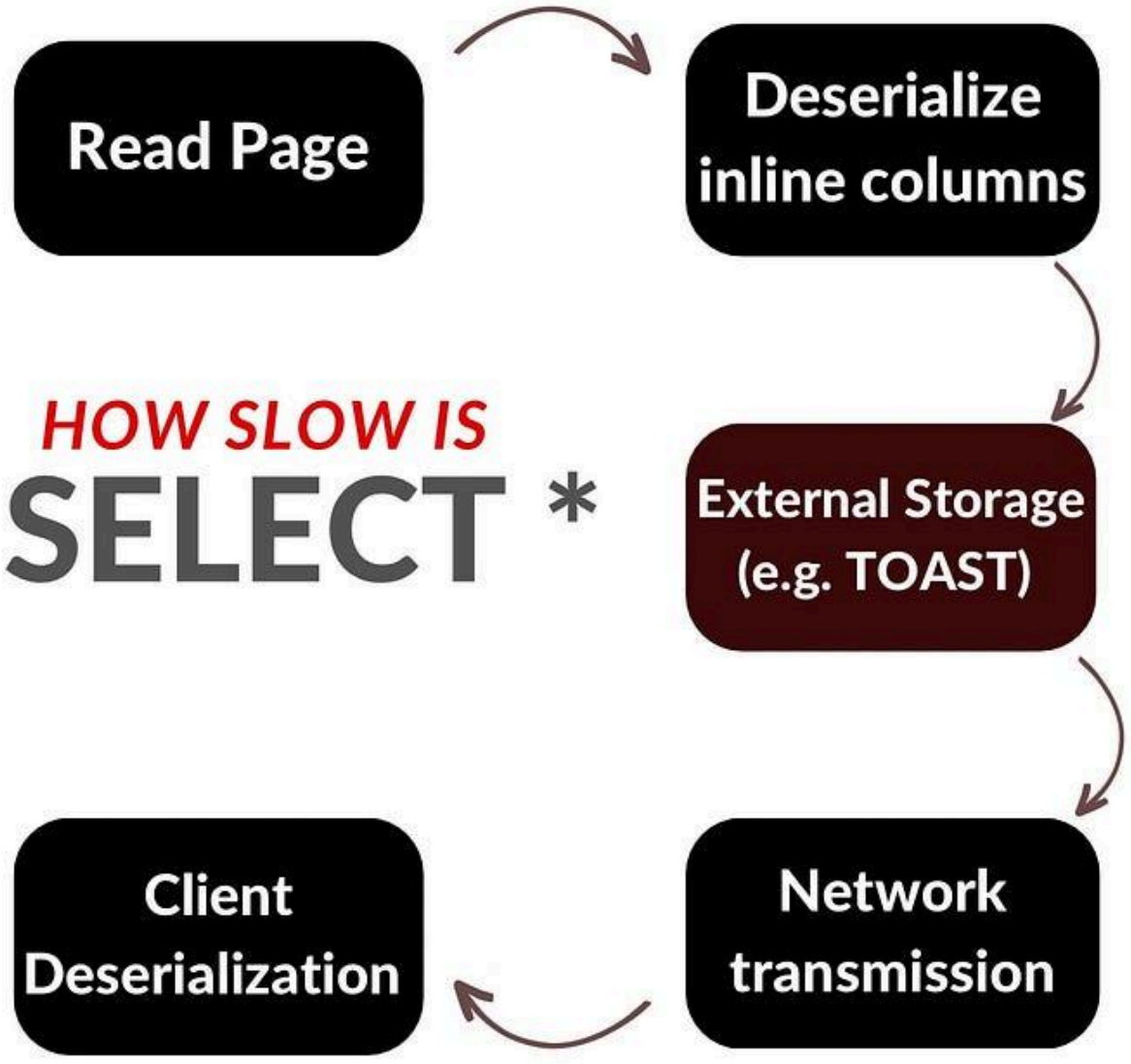
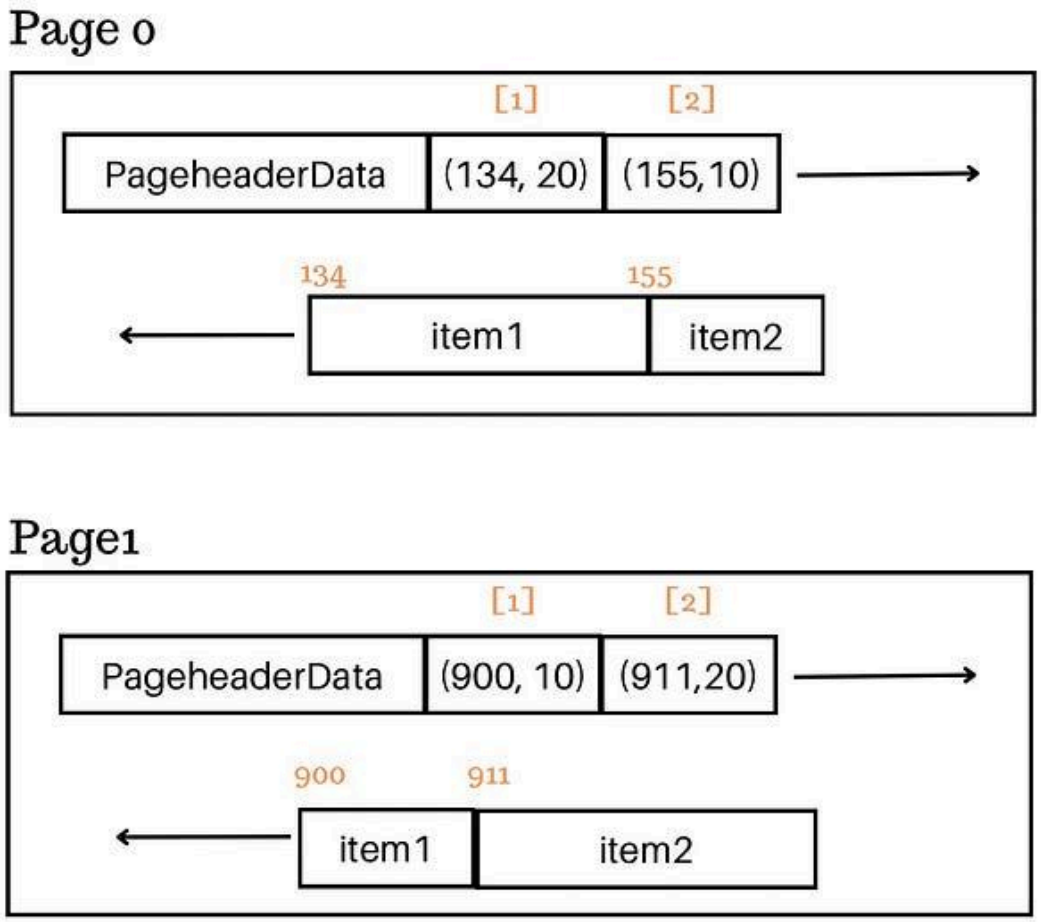
How Slow is Select *?



In a row-store database engine, rows are stored in units called pages. Each page has a fixed header and contains multiple rows, with each row having a record header followed by its respective columns. For instance, consider the following example in PostgreSQL:

Postgres Page



When the database fetches a page and places it in the shared buffer pool, we gain access to all rows and columns within that page. So, the question arises: if we have all the columns readily available in memory, why would SELECT * be slow and costly? Is it really as slow

as people claim it to be? And if so why is it so? In this post, we will explore these questions and more.

Kiss Index-Only Scans Goodbye

Using SELECT * means that the database optimizer cannot choose index-only scans. For example, let's say you need the IDs of students who scored above 90, and you have an index on the grades column that includes the student ID as a non-key, this index is perfect for this query.

However, since you asked for all fields, the database needs to access the heap data page to get the remaining fields increasing random reads resulting in far more I/Os. In contrast, the database could have only scanned the grades index and returned the IDs if you hadn't used SELECT *.

Deserialization Cost

Deserialization, or decoding, is the process of converting raw bytes into data types. This involves taking a sequence of bytes (typically from a file, network communication, or another source) and converting it back into a more structured data format, such as objects or variables in a programming language.

When you perform a SELECT * query, the database needs to deserialize all columns, even those you may not need for your specific use case. This can increase the computational overhead and slow down query performance. By only selecting the necessary columns, you can reduce the deserialization cost and improve the efficiency of your queries.

Not All Columns Are Inline

One significant issue with SELECT * queries is that not all columns are stored inline within the page. Large columns, such as text or blobs, may be stored in external tables and only retrieved when requested

(Postgres TOAST tables are example). These columns are often compressed, so when you perform a SELECT * query with many text fields, geometry data, or blobs, you place an additional load on the database to fetch the values from external tables, decompress them, and return the results to the client.

Network Cost

Before the query result is sent to the client, it must be serialized according to the communication protocol supported by the database. The more data needs to be serialized, the more work is required from the CPU. After the bytes are serialized, they are transmitted through TCP/IP. The more segments you need to send, the higher the cost of transmission, which ultimately affects network latency.

Returning all columns may require deserialization of large columns, such as strings or blobs, that clients may never use.

Client Deserialization

Once the client receives the raw bytes, the client app must deserialize the data to whatever language the client uses, adding to the overall processing time. The more data is in the pipe the slower this process.

Unpredictability

Using SELECT * on the client side even if you have a single field can introduce unpredictability. Think of this example, you have a table with one or two fields and your app does a SELECT * , blazing fast two integer fields.

However, later the admin decided to add an XML field, JSON, blob and other fields that are populated and used by other apps. While your code did not change at all, it will suddenly slow down because it is now picking up all the extra fields that your app didn't need to begin with.

Summary

In conclusion, a SELECT * query involves many complex processes, so it's best to only select the fields you need to avoid unnecessary overhead. Keep in mind that if your table has few columns with simple data types, the overhead of a SELECT * query might be negligible. However, it's generally good practice to be selective about the columns you retrieve in your queries.

Thank you.